

Desenvolvimento de Backend para Gestão Fitness (FitTrack API)

Autores: Douglas Rodrigues e Mauricio T. Welter
Curso: Análise e Desenvolvimento de Sistemas (ADS)
Instituição: Unisenac

1. Introdução

O presente trabalho descreve o desenvolvimento da FitTrack API, uma aplicação backend de alta performance projetada para centralizar e otimizar a gestão de treinos físicos. Em um mundo cada vez mais voltado para o monitoramento de dados de saúde, a necessidade de sistemas que ofereçam integridade de dados, segurança e rapidez é fundamental. Este projeto aplica os princípios de **Clean Architecture** e **Domain-Driven Design (DDD)** para criar uma estrutura escalável e de fácil manutenção, utilizando o framework NestJS como base tecnológica.

2. Definição do Problema

O acompanhamento de atividades físicas muitas vezes é feito de forma fragmentada, seja em papéis, aplicativos simples, sem sincronização ou planilhas. A ausência de um backend centralizado e seguro impede que o usuário tenha um histórico confiável e acessível. Além disso, sistemas que gerenciam grandes volumes de dados de múltiplos usuários frequentemente sofrem com latência nas consultas de listagem, o que prejudica a experiência do usuário (UX). O problema reside na falta de uma solução que una segurança (autenticação), performance (cache) e organização arquitetural robusta.

3. Objetivos e Solução

A solução proposta é a **FitTrack API**. O objetivo principal é fornecer um ecossistema backend onde usuários possam se cadastrar com segurança e gerenciar seus treinos de forma individualizada. A ideia resolve o problema ao oferecer:

- **Segurança:** Garantia de que os dados de treino pertencem apenas ao seu criador através de autenticação via JWT.
- **Performance:** Redução drástica no tempo de resposta através de estratégias de cache inteligente.
- **Escalabilidade:** Uma arquitetura que permite adicionar novas funcionalidades (como módulos de nutrição ou social) sem comprometer o código existente.

4. Requisitos do Sistema

4.1. Requisitos Funcionais

- **RF01 - Cadastro de Usuário:** O sistema deve permitir que novos usuários se cadastrem com nome, e-mail e senha.
- **RF02 - Autenticação:** O sistema deve validar as credenciais do usuário e retornar um token JWT válido.
- **RF03 - CRUD de Treinos:** O usuário autenticado deve poder criar, ler, atualizar e excluir seus registros de treino.
- **RF04 - Persistência de Dados:** Todos os dados devem ser armazenados de forma relacional no banco de dados PostgreSQL.

4.2. Requisitos Não Funcionais

- **RNF01 - Performance (Cache):** Listagens de treinos devem ser servidas via cache em memória para reduzir o acesso ao disco.
- **RNF02 - Segurança:** Senhas devem ser armazenadas utilizando algoritmos de hash (bcrypt).
- **RNF03 - Arquitetura:** O código deve seguir o padrão de arquitetura em camadas (Presentation, Application, Infrastructure).
- **RNF04 - Disponibilidade:** A API deve seguir os princípios RESTful para garantir interoperabilidade com diferentes frontends.

5. Regras de Negócio

- **RN01 - Unicidade de E-mail:** Não é permitido o cadastro de dois usuários com o mesmo endereço de e-mail.
- **RN02 - Propriedade de Dados:** Um usuário jamais pode visualizar, editar ou excluir o treino que pertença a outro usuário.
- **RN03 - Invalidação de Cache:** Sempre que um treino for criado, alterado ou excluído, o cache de listagem do usuário deve ser limpo para garantir a consistência.

6. Arquitetura e Tecnologias Utilizadas

6.1. Tecnologias

- **NestJS:** Escolhido por ser um framework opinativo que utiliza TypeScript e injeção de dependência nativa, o que facilita a criação de código testável e modular.
- **PostgreSQL & TypeORM:** O PostgreSQL foi escolhido pela sua robustez e suporte a transações ACID. O TypeORM facilita a manipulação do banco via objetos TypeScript, automatizando migrações.

- **Redis/Memory Cache:** Utilizado para o requisito de performance, evitando consultas desnecessárias ao banco em dados que não mudam frequentemente.
- **Passport JWT:** Padrão de mercado para autenticação stateless em APIs.

6.2. Estrutura de Camadas

O projeto divide-se em:

- **Camada de Apresentação (Controllers):** Gerencia as rotas, os DTOs de entrada e a resposta para o cliente.
- **Camada de Aplicação (Services):** Contém o "coração" da aplicação, onde as Regras de Negócio são aplicadas.
- **Camada de Infraestrutura:** Lida com o banco de dados, provedores de cache e segurança externa.

7. Especificação das APIs (Endpoints)

Método	Endpoint	Descrição	Acesso
POST	/users	Cria um novo usuário.	Público
POST	/auth/login	Realiza login e gera Token.	Público
GET	/workouts	Lista treinos do usuário (Cacheado).	Privado
POST	/workouts	Cria um novo treino.	Privado

8. Metodologia de Desenvolvimento e Uso de IA

O desenvolvimento seguiu a metodologia ágil, com uso crítico de ferramentas de IA para aceleração de código. A IA foi utilizada para gerar esqueletos de testes unitários e sugerir estruturas de DTOs. A intervenção humana foi crucial para refinar a lógica de segurança, garantindo que o AuthGuard bloqueasse acessos indevidos e que o CacheInterceptor funcionasse por usuário e não de forma global.

9. Conclusão

A FitTrack API cumpre os requisitos de um sistema backend de nível profissional. Ao integrar autenticação JWT, cache em memória e uma arquitetura limpa, o projeto não apenas resolve o problema de organização de dados fitness, mas o faz de forma escalável e performática. Este trabalho demonstra a maturidade técnica necessária para o desenvolvimento de sistemas modernos no ecossistema Node.js.

10. Referências Bibliográficas

- **NESTJS.** Documentation. Disponível em: <https://docs.nestjs.com>. Acesso em: 11 mai. 2026.
- **TYPEORM.** Data Mapper and ActiveRecord documentation. Disponível em: <https://typeorm.io>. Acesso em: 11 mai. 2026.
- **POSTGRESQL.** Official Documentation. Disponível em: <https://www.postgresql.org/docs>. Acesso em: 11 mai. 2026.
- **OWASP.** JSON Web Token Cheat Sheet. Disponível em: <https://cheatsheetseries.owasp.org>. Acesso em: 11 mai. 2026.